# 6.5 Maximum Likelihood Hypotheses for Predicting Probabilities – Notes

---

## 1. Motivation

- In regression with continuous-valued outputs (Section 6.4): **Maximum Likelihood** under Gaussian noise ⇒ minimize **squared error**.
- In classification with probabilistic binary outputs: **Maximum Likelihood** under Bernoulli noise ⇒ minimize **cross-entropy error**.
- This section shows why cross-entropy is the natural criterion for learning probabilistic classifiers (e.g., logistic regression, neural networks).

---

## 2. Problem Setting

- Target function: $f : X \to \{0, 1\}$ , nondeterministic (probabilistic).
- Training data: $D = \{(x_1, d_1), \ldots, (x_m, d_m)\},\ d_i \in \{0, 1\}$ .
- Hypothesis: $h(x) \in [0, 1]$ , interpreted as $P(f(x) = 1)$ .
- Example applications:
- Medical diagnosis: survival probability given symptoms.
- Loan repayment: probability of default given credit history.

---

## 3. Likelihood of Data

- For a single data point $(x_i, d_i)$ :
- If $d_i = 1 : P(d_i \mid h, x_i) = h(x_i)$ .
- If $d_i = 0 : P(d_i \mid h, x_i) = 1 - h(x_i)$ .
- Unified form:

$$P(d_i \mid h, x_i) = h(x_i)^{d_i}(1 - h(x_i))^{1-d_i}$$

- For all data (independent examples):

$$P(D \mid h) = \prod_{i=1}^{m} h(x_i)^{d_i}(1 - h(x_i))^{1-d_i}$$

---

## 4. Connection to Binomial Distribution

- This expression generalizes the **Binomial distribution**:

• Binomial assumes *identical probabilities* for all trials.
• Here, each instance $x_i$ can have its own probability $h(x_i)$ .
• Analogy: $m$ coin flips, each coin $i$ with bias $h(x_i)$ , outcomes given by $d_1, \ldots, d_m$ .
• Assumption: coin flips (examples) are **independent**.

---

## 5. Log-Likelihood and Cross-Entropy

• Take log:

$$\ell(h) = \sum_{i=1}^{m} \left[ d_i \log h(x_i) + (1 - d_i) \log(1 - h(x_i)) \right]$$

• Maximizing $\ell(h)$ = maximizing likelihood.
• Equivalent to minimizing the **negative log-likelihood**:

$$-\ell(h) = -\sum_{i=1}^{m} \left[ d_i \log h(x_i) + (1 - d_i) \log(1 - h(x_i)) \right]$$

• This is the **cross-entropy error function**.

---

## 6. Comparison with Regression Case

• **Continuous outputs** + Gaussian noise → ML criterion = **minimize squared error**.
• **Binary probabilistic outputs** + Bernoulli distribution → ML criterion = **minimize cross-entropy**.

---

## 7. Practical Implications

• Cross-entropy is standard in:
• Logistic regression.
• Neural networks for classification (softmax + cross entropy).
• Any task predicting probabilities of discrete outcomes.
• Just like squared error in regression, **cross-entropy has a Bayesian justification**.

---

## 8. Teaching Tips

• Use **coin flip analogy**: each instance = biased coin, probability $h(x_i)$ .
• Relate log-likelihood to entropy:
• Entropy: $-\sum p_i \log p_i$ .
• Cross-entropy: $-\sum d_i \log h(x_i)$ .
• Key point: cross-entropy isn't arbitrary; it's the ML solution under Bernoulli noise.

---

✅**Main Takeaway**:
- Regression tasks (Gaussian noise): **Least-Squares = ML**.
- Classification tasks (Bernoulli noise): **Cross-Entropy = ML**.
Both are special cases of the same Bayesian principle.

---

# 6.5.1 Gradient Search to Maximize Likelihood in a Neural Net

---

### 1. Background

- From Section 6.5: ML criterion for probabilistic classifiers = maximize log-likelihood.
- Equivalent to minimizing cross-entropy.
- Now derive gradient ascent weight update rule for neural networks.

---

### 2. Notation

- Log-likelihood:

$$G(h, D) = \sum_{i=1}^{m} \left[ d_i \log h(x_i) + (1 - d_i) \log(1 - h(x_i)) \right]$$

- Hypothesis $h$ represented by network weights $w_{jk}$ .

---

### 3. Gradient Ascent Rule

- Compute gradient: partial derivatives of $G(h, D)$ wrt weights.
- For a sigmoid unit network:

$$\Delta w_{jk} = \eta \sum_{i=1}^{m} (d_i - h(x_i)) x_{ijk}$$

where:
- $d_i$ = target output (0 or 1).
- $h(x_i)$ = predicted probability.
- $x_{ijk}$ = input to weight $w_{jk}$ .
- $\eta$ = learning rate.

---

### 4. Intuition

- If model underestimates when $d = 1 \rightarrow$ increase weights.

- If model overestimates when $d = 0 \rightarrow$ decrease weights.
- Always adjusts in direction that increases log-likelihood.

---

## 5. Comparison with Backpropagation (Squared Error)

- Backprop with squared error (Gaussian assumption):

$$\Delta w_{jk} = \eta \sum_{i=1}^{m} (d_i - h(x_i)) h(x_i)(1 - h(x_i)) x_{ijk}$$

- Difference: extra factor $h(x_i)(1 - h(x_i))$ from sigmoid derivative.
- In cross-entropy training, this term cancels → simpler update rule.

---

## 6. Summary

- Both rules converge to ML hypotheses but under different assumptions:
- **Squared error** → ML under Gaussian noise.
- **Cross entropy** → ML under Bernoulli noise.
- Explains why modern classification networks use **cross entropy + gradient ascent (or gradient descent on negative log-likelihood)**.

---

✅**Teaching Tip**: Show the cancellation of the sigmoid derivative term as the key simplification when using cross entropy.